Sharelock OZmes & Web3son

Gourge Julien & Libert Corentin

April 12, 2026

Dans le cadre de ce projet, vous avez l'honneur de prendre part aux aventures de Sharelock OZmes [1] et du docteur Web3son [2], dignes héritiers de Sherlock Holmes [3] et du docteur Watson [4]. Mais en cette saison capricieuse, le célèbre détective est cloué au lit, terrassé par la grippe, incapable de parler ou de se mouvoir.

Avant de succomber à la maladie, Sharelock a pourtant résolu l'énigme et consigné sa réponse sur un papier, désormais enfermé dans son coffre personnel. Hélas, lui seul en connaît la combinaison, et le commissaire de Londres s'impatiente. Acculé, Web3son fait appel à vous : informaticiens en devenir, mais aussi détectives amateurs. Il vous confie les notes de Sharelock et les éléments de l'enquête. À vous de jouer !

À leur lecture, vous comprenez que le coffre s'ouvre à l'aide d'une phrase secrète... absente des notes. À la place, vous découvrez des schémas et explications d'un système énigmatique, une liste de transactions, ainsi que cette indication :

> *«La solution est à la vue de tous pour celui qui sait regarder. »*

Après réflexion, l'évidence s'impose : le système décrit n'est autre qu'une forme simplifiée de Blockchain [5], semblable à ce fameux Bitcoin [6] dont vous avez entendu parler. Il vous faut donc le reconstituer pour traiter la liste de transactions. Une fois l'opération achevée, l'état final révélera la phrase secrète.

Vous vous tournez alors vers Web3son, et clamé d'un ton assuré :

> *«Élémentaire, mon cher Web3son ! Sharelock a dissimulé la clé sous nos yeux. Recréons son système, et la vérité apparaîtra. »*

Rassuré, Web3son s'éclipse prévenir le commissaire. De votre côté, vous prenez place devant votre ordinateur, le carnet ouvert à côté de vous. Il est temps de se mettre au travail !

---

[1] `https://en.wikipedia.org/wiki/Oz_(programming_language)`
[2] `https://fr.wikipedia.org/wiki/Web3`
[3] `https://fr.wikipedia.org/wiki/Sherlock_Holmes`
[4] `https://fr.wikipedia.org/wiki/Docteur_Watson`
[5] `https://en.wikipedia.org/wiki/Blockchain`
[6] `https://bitcoin.org/fr/`

1 Consignes générales

Avant de commencer ce projet, **veuillez lire en attentivement les consignes suivantes:**

- Le projet est a réaliser **par groupe de 2 étudiants** (vous n'êtes pas obligé de faire le TP dans le même local).
- **Respectez les spécifications** données pour les fonctions à implémenter. La correction sera en partie automatisée et les fonctions qui ne respectent pas les spécifications ne seront donc pas prises en compte.
- Le **plagiat est strictement interdit**. Il est autorisé de discuter du projet entre groupes et de partager des idées, mais il est interdit de partager du code. Tout code partagé ou copié sera considéré comme du plagiat et sanctionné en conséquence. Vos codes seront vérifiés par un algorithme anti-plagiat.
- **L'utilisation de l'IA est autorisée pour la forme du rapport et la correction orthographique**. Elle n'est pas autorisée pour la génération du fond du rapport, et pas recommandée pour la génération de code. Tous rapports et/ou codes semblant avoir été générés par l'IA pourra faire l'objet de questions supplémentaires sous la forme d'un oral afin de déterminer si vous comprenez votre travail. **Toute utilisation de l'IA doit être mentionnée.**
- Le projet (**code + rapport**) est à remettre sur INGInious pour le **10 mai 2026 à 23h59** au plus tard. Veuillez vous référer à la section "Consignes de soumissions". Les soumissions non conformes aux consignes seront pénalisées. Comme pour chaque projet, nous vous recommandons de ne pas soumettre en dernière minute afin d'éviter les problèmes de connexion ou de serveur. **Ne prenez aucun risque, prenez de l'avance !**

2 Marche à suivre

Dans le cadre de ce projet, il vous est demandé d'implémenter une version simplifiée d'une blockchain, d'exécuter les transactions fournies et de décrypter l'état final obtenu pour trouver la phrase secrète. Vous devez également implémenter deux extensions au choix parmi celles proposées (voir section 3).

Afin de réaliser ce projet avec succès, nous vous conseillons vivement de suivre dans l'ordre les étapes suivantes.

1. Implémentation du système (blockchain).
2. Implémentation de la fonction de décodage.
3. Implémentation des extensions.

2.1 Implémentation du système (blockchain)

Une blockchain [7] est un système décentralisé capable de répertorier des transactions entre utilisateurs de façon immuable et transparente.

Nous vous demandons d'implémenter une version simplifiée d'un tel système et suivant les spécifications données. Un squelette des fonctions à implémenter, comprenant les signatures des fonctions ainsi que leurs spécifications (pré- et post-conditions), vous est fourni. **Il vous est demandé de ne pas les modifier**, cela entraînant des problèmes pour la correction. Vous êtes cependant libre de créer de nouvelles fonctions et de les utiliser dans les fonctions à implémenter. Seules les fonctions demandées seront exécutées.

2.1.1 Qu'est ce qu'une blockchain

Une blockchain est une chaîne de blocs, liés un à un. Chaque bloc est composé de transactions, ce que nous considérerons comme l'unité de base dans ce projet.

Une **transaction** représente un échange entre 2 utilisateurs. Dans ce projet, elle sera représentée par un enregistrement (*record*) ayant le format suivant:

$$\begin{aligned}\langle transaction\rangle = \{\ & nonce : \langle integer\rangle,\\ & block_number : \langle integer\rangle,\\ & hash : \langle integer\rangle,\\ & sender : \langle integer\rangle,\\ & receiver : \langle integer\rangle,\\ & value : \langle integer\rangle,\\ & effort : \langle integer\rangle,\\ & max_effort : \langle integer\rangle\ \}\end{aligned}$$

Avec :

- *nonce*, un entier représentant le nombre de transactions qu'un utilisateur a envoyées. Il commence à 1 pour la première transaction et est incrémenté de 1 à chaque nouvelle transaction de l'utilisateur (la n^{eme} transaction d'un utilisateur a son *nonce* égal à $n+1$). Il est invalide s'il est négatif ou que $nonce \neq nonce_{precedent} + 1$.
- *block_number*, un entier représentant le numéro du bloc dans lequel la transaction doit être incluse.
- *hash*, un entier (non-nul) représentant le hash de la transaction. Il est invalide s'il est nul ou qu'il ne correspond pas au résultat de la fonction de hachage (voir fonctions de hachage 2.1.2).

[7] https://en.wikipedia.org/wiki/Blockchain

- *sender*, un entier représentant l'identifiant de l'envoyeur de la transaction.
- *receiver*, un entier représentant l'identifiant du receveur de la transaction.
- *value*, un entier représentant une valeur non négative qui est envoyée par l'envoyeur à destination du receveur. Elle est invalide, si négative.
- $effort$, un entier positif représentant l'effort calculatoire nécessaire pour exécuter cette transaction. Cette valeur correspond au résultat de la fonction de calcul d'effort(voir algorithme du calcul de l'effort 2.1.3).
- *max_effort, un entier positif représentant l'effort maximal que l'envoyeur est prêt à fournir pour exécuter la transaction. Si l'*$effort$ *est supérieur à* max_effort la transaction ne sera pas exécutée. Il est invalide s'il est négatif.

Un **bloc** est un ensemble de transactions. Il est représenté par un enregistrement (*record*) ayant le format suivant:

$$\begin{aligned}\langle block\rangle = \{\ & number : \langle integer\rangle,\\ & previousHash : \langle integer\rangle,\\ & transactions : \langle list_of_transactions\rangle,\\ & hash : \langle integer\rangle\ \}\end{aligned}$$

avec :

- *number*, un entier représentant le numéro du bloc dans la chaîne. Le premier bloc a pour numéro 0, le bloc suivant a pour numéro 1, etc. Il est invalide s'il est négatif ou que $number \neq number_{precedent} + 1$.
- $previousHash$, un entier représentant le hash du bloc précédent dans la chaîne. Le premier bloc a pour $previousHash$ la valeur 0. Il est invalide s'il est négatif ou que $previousHash \neq hash_{precedent}$.
- *transactions*, une liste de transactions. Elle est invalide si elle est nulle ou que l'une des transactions qu'elle contient est invalide.
- *hash*, un entier représentant le hash du bloc. Il est invalide s'il est nul ou qu'il ne correspond pas au résultat de la fonction de hachage d'un bloc (voir fonctions de hachage 2.1.2).

Enfin, l'unité la plus haute de ce système est la **blockchain**, qui est une liste de blocs :

$\langle blockchain\rangle ::= \text{nil} \mid \langle block\rangle$ **'|'** $\langle blockchain\rangle$

La liste des blocs est ordonnée par numéro de bloc croissant, c'est-à-dire que le bloc de numéro n est suivi du bloc de numéro $n + 1$.

2.1.2 Fonctions de hachage

Pour cette partie, vous devez implémenter deux fonctions de hachage. Nous vous recommandons de les implémenter en premier, car elles sont utilisées tout au long du projet.

Une fonction de hachage permet de générer une valeur appelée "hash" à partir des caractéristiques d'une transaction ou d'un bloc. Cette valeur est ensuite utilisée pour les identifier et s'assurer de leur intégrité. Les algorithmes de hachage sont la plupart du temps complexes à implémenter, c'est pourquoi nous vous demandons d'implémenter une version très simplifiée d'un tel algorithme. Dans ce projet, il existe deux fonctions de hachage différentes : une pour les transactions et une pour les blocs.

Pour calculer le hash d'une **transaction** :

$$TransitionHash = (nonce + sender + receiver + value) \mod 10^6$$

Pour calculer le hash d'un **bloc** :

$$BlockHash = (number + previousHash + \sum_{i=1}^{\#transactions} hashTransaction_i) \mod 10^6$$

2.1.3 Calcul de l'effort

Vous devez implémenter une fonction de calcul de l'effort nécessaire pour effectuer une transaction. L'effort d'une transaction est calculé de la manière suivante :

$$effort = \sum_{i=0}^{len(value)-1} 2^i$$

Dans ce cas ci, $len(value)$ représente le nombre de chiffres dans la valeur de la transaction.

Par exemple, l'effort d'une transaction avec une valeur de 500 est égal à $2^0 + 2^1 + 2^2 = 7$.

2.1.4 Validation d'une transaction

Vous devez également être capable de décider si une transaction est valide ou non. Pour qu'une transaction soit valide, vous devez vérifier les points suivants:

- le *nonce* doit être égal au *nonce* de la dernière transaction envoyée par **cet** utilisateur + 1,
- le *hash* de la transaction correspond au résultat de la fonction de hachage,
- le sender a **suffisamment de fonds** pour effectuer la transaction,
- le *value* de la transaction est positif ou nul,

- le *max_ effort de la transaction est positif,*
- *l'effort de la transaction (calculé par la fonction de calcul de l'effort) ne dépasse pas son* max_effort

Vous ne devez pas vérifier le format et les types des features du record, nous supposons que les transactions sont déjà parsées et que les champs sont déjà convertis dans le bon type et sont tous existants.

Toute transaction qui ne respecte pas l'une de ces conditions doit être considérée comme invalide et ne doit pas être ajoutée à un bloc.

2.1.5 Validation d'un bloc

De la même manière, vous devez être capable de décider si un bloc est valide ou non. Pour qu'un bloc soit valide, vous devez vérifier les points suivants:

- le *number* du bloc doit être égal au *number* du bloc précédent + 1,
- le *previousHash* du bloc doit être égal au *hash* du bloc précédent,
- le *hash* du bloc doit correspondre au résultat de la fonction de hachage d'un bloc,
- toutes les transactions du bloc doivent être valides,
- **la somme des efforts de toutes les transactions du bloc ne doit pas dépasser l'effort maximal d'un block qui est de** 300**.** Si ajouter une transaction à un bloc entraîne un dépassement de ce seuil, alors cette transaction ne doit pas être ajoutée au bloc.

2.1.6 Gestion de l'état

Vous devez être capable de représenter, à tout instant t, le solde de chaque utilisateur présent dans la blockchain (de manière analogue au solde d'un compte bancaire) ainsi que le dernier *nonce* utilisé. Ces informations sont nécessaires pour valider les transactions : une transaction doit être refusée si l'utilisateur émetteur ne dispose pas de fonds suffisants et si son *nonce* n'est pas correct.

Le nombre d'utilisateurs n'est pas connu à l'avance, lorsqu'un utilisateur connu envoie un montant à un utilisateur inconnu, vous devez considérer que ce dernier devient un nouvel utilisateur de la blockchain, avec un solde initial égal au montant reçu.

Pour répondre à ces exigences, vous devez maintenir une représentation de l'état de la blockchain. Voici comment vous devez le représenter :

$$\begin{aligned}\langle state\rangle = \{\ & address1 : user(balance : int, nonce : int),\\ & address2 : user(balance : int, nonce : int),\\ & \ldots\}\end{aligned}$$

avec :

- $addressX$, un entier représentant l'identifiant d'un utilisateur.
- $balanceX$, un entier représentant le solde de l'utilisateur $addressX$.
- $nonceX$, un entier représentant le dernier *nonce* utiliser par l'utilisateur $addressX$.

Cet état est la représentation minimale que vous devez maintenir. Si vous le souhaitez, vous pouvez faire évoluer le record user pour y ajouter d'autres informations que vous jugez utiles pour le projet.

Pour démarrer, vous avez besoin de connaître le solde d'au moins un utilisateur (sinon aucune transaction ne sera valide). Pour ce faire, nous vous fournissons un record représentant l'état initial de la blockchain dans un bloc spécial, appelé le bloc génésis. Cet état initial est représenté de la manière suivante :

$$\langle genesis \rangle = \{\ address1 : balance1, address2 : balance2, \ldots\}$$

Vous devez utiliser cet état comme point de départ pour construire l'état de la blockchain.

2.1.7 Executer la blockchain

Maintenant que vous avez implémenté les fonctions nécessaires pour créer la blockchain, vous devez être capable de l'exécuter (c'est-à-dire de la créer et d'y appliquer les transactions) avec les transactions fournies. Pour ce faire, vous devez implémenter la procédure `{ExecuteBlockchain GenesisState Transactions FinalState FinalBlockchain}`. Cette procédure prend en entrée 4 arguments :

- $GenesisState$, un record représentant l'état initial de la blockchain (comme expliqué dans la section 2.1.6).
- $Transactions$, la liste de toutes les transactions à exécuter (attention, ces transactions ne contiennent pas l'effort nécessaire pour les exécuter, vous devez le calculer vous même et l'ajouter à la transaction). Ces transactions sont triées par ordre de bloc croissant et par ordre dans lequel vous devez les traiter. Vous pouvez voir cette liste dans le fichier transactions.txt dans le dossier data du projet.
- $FinalState$, une variable non initialisée que vous devez assigner à l'état final de la blockchain après exécution de toutes les transactions.
- $FinalBlockchain$, une variable non initialisée que vous devez assigner à la blockchain finale après exécution de toutes les transactions.

Hint : Vous êtes libre d'implémenter les fonctions que vous souhaitez en plus des deux fonctions obligatoires (ExecuteBlockchain et Decode). Voici cependant

quelques conseils (que vous n'êtes pas obligé de suivre) pour implémenter le projet. Nous vous recommandons de commencer par regrouper les transactions par bloc, puis d'ajouter les transactions valides au bloc correspondant en respectant les conditions de validation expliquées précédemment. Faites également attention à ne pas dépasser l'effort maximal d'un bloc. Une fois que vous avez créé le bloc, vous pouvez l'ajouter à la blockchain et ainsi de suite jusqu'à ce que vous ayez traité toutes les transactions. Vous devez également mettre à jour l'état de la blockchain à chaque fois que vous appliquez une transaction valide.

2.2 Fonction de déchiffrage

Maintenant que vous avez implémenté le système de blockchain, vous devez être capable de l'exécuter avec les transactions fournies et de découvrir la phrase secrète pour ouvrir le coffre de Sharelock. Pour ce faire, vous devez implémenter une fonction de décodage. Cette fonction prend en entrée la blockchain finale (après exécution de toutes les transactions) et retourne la phrase secrète.

Voici la signature de cette fonction : `{Decode Blockchain}` qui prend en entrée la blockchain et retourne la phrase secrète.

Pour implémenter la fonction de décode, vous devez suivre les étapes suivantes :

```
Pour chaque bloc de la blockchain :
    Récupérer le hash du bloc
    Pour chaque PAIRE de chiffres consécutifs X dans le hash :
        Nombre = X mod 37
        Si Nombre est inférieur à 10,
            Nombre = 36
        Convertir Nombre en lettre en utilisant le tableau de Sharelock
    Ajouter tous les caractères obtenus à la phrase secrète
Retourner la phrase secrète
```

Par exemple, si le hash d'un bloc est égal à 284110 on obtient :

- $28 \mod 37 = 28$, qui correspond à la lettre "S" dans le tableau de Sharelock.
- $51 \mod 37 = 14$, qui correspond à la lettre "E" dans le tableau de Sharelock.
- $10 \mod 37 = 10$, qui correspond à la lettre "A" dans le tableau de Sharelock.

On obtient donc la phrase secrète "SEA" à partir de ce hash.

Si le hash a un nombre impair de chiffres, le dernier chiffre ne doit pas être pris en compte.

Nombre	**Lettre**	**Nombre**	**Lettre**	**Nombre**	**Lettre**	**Nombre**	**Lettre**
10	a	11	b	12	c	13	d
14	e	15	f	16	g	17	h
18	i	19	j	20	k	21	l
22	m	23	n	24	o	25	p
26	q	27	r	28	s	29	t
30	u	31	v	32	w	33	x
34	y	35	z	36	*space*		

Table 1: Note de Sharelock pour le décodage de la phrase secrète (oui, Sharelock a une façon très étrange de faire ses tableaux...).

3 Extensions

Vous devez implémenter au moins deux des trois extensions suivantes. La phrase secrète n'aura plus de sens avec les extensions. C'est pour cette raison que nous vous demandons d'envoyer le fichier BaseModule.oz sans les extensions et un fichier ExtensionsName.oz par extension implémentée. Cela nous permettra de tester votre code avec et sans les extensions.

3.1 Denylist

Le principe de cette extension est d'ajouter une denylist d'utilisateurs à la blockchain. Les utilisateurs présents dans cette denylist sont considérés comme malveillants et toutes les transactions les impliquant (en tant que sender) sont considérées comme invalides et ne doivent pas être ajoutées à un bloc. Pour qu'un utilisateur soit ajouté à la denylist, il doit envoyer au moins trois transactions au sein d'un même bloc (transactions valides ou non). Une fois qu'un utilisateur est ajouté à la denylist, il doit y rester pour le reste de la blockchain. Après l'ajout à la denylist, les transactions envoyées par cet utilisateur ne doivent plus être prises en compte (les transactions déjà envoyées par cet utilisateur avant son ajout à la denylist sont prises en compte si elles sont valides).

3.2 L'effort n'est pas gratuit

Pour le moment, vous calculez l'effort nécessaire pour exécuter une transaction, mais cet effort a un coût pour l'utilisateur. Le but de cette extension est de prendre en compte ce coût dans la validation de la transaction. En supposant que le coût de l'effort est de **une unité monétaire par unité d'effort** (un effort de 12 coûtera donc 12 au sender), vous devez vérifier que l'utilisateur qui fait la transaction a suffisamment de fonds sur son compte pour payer le coût de l'effort nécessaire pour exécuter la transaction. Si ce n'est pas le cas, la transaction doit être refusée, sinon elle est exécutée et le coût de l'effort est déduit du solde de l'utilisateur en plus du montant de la transaction.

3.3 Féru de stats

Cette extension a pour but de vous faire faire des statistiques sur la blockchain. Vous devez au moins implémenter les fonctions suivantes :

- `{BiggestSenders Blockchain n}` : retourne une liste des n utilisateurs ayant fait le plus de transactions (qui ont été validées).
- `{ReceivedMost Blockchain n}` : retourne une liste des n utilisateurs ayant reçu la plus grande valeur totale de transactions.
- `{MoneyPeak Blockchain}` : retourne l'utilisateur qui a eu le solde le plus élevé à un moment donné de la blockchain, ainsi que ce solde.

Remarques:

- En cas d'égalité, les utilisateurs sont classés par ordre alphabétique.
- Si n est supérieur au nombre d'utilisateurs, vous devez retourner la liste de tous les utilisateurs classés selon les critères précédents.
- Si n est négatif ou égal à 0, vous devez retourner -1.

4 Rapport

Le rapport doit avoir une longueur de **maximum 4 pages** (ou 5 si vous décidez d'implémenter les 3 extensions). **Toute page supplémentaire sera pénalisée.** Le rapport doit être structuré de la manière suivante :

1. Une page pour expliquer les choix que vous avez faits pour implémenter le système de blockchain, principalement les fonctions RunBlockchain et AppendToBlockchain.
2. Une page pour présenter l'état final de la blockchain, c'est-à-dire, le solde de chaque utilisateur et son dernier nonce, et une représentation de la blockchain finale (la liste des blocs). Cette représentation doit montrer chaque bloc avec son numéro, hash, previousHash et le hash des transactions (n'affichez pas les autres champs des transactions pour ne pas surcharger la figure). Enfin, vous devez afficher la phrase secrète obtenue à partir de la blockchain finale.
3. Une page par extension que vous avez choisie où vous expliquez comment vous avez implémenté l'extension, les choix que vous avez faits, ainsi que l'état final de la blockchain, la liste des blocs et la phrase secrète obtenue.

N'oubliez pas de mentionner le NOM, PRÉNOM et NOMA de chaque étudiant du groupe.

5 Consignes de soumissions

Votre solution doit suivre le format du squelette de code fourni, référez-vous au fichier $CONSIGNE.md$. Vous devez soumettre le fichier ZIP (créé en utilisant make zip) contenant votre code et votre rapport sur INGInious avant la date limite.